

EL2004 -Electrical Network Analysis

Complex Engineering Activity Report

Fall-2025



Auto-Track (Arduino-Based Intelligent Autonomous Car with Hybrid Navigation & Bluetooth Control)

Submitted To

DR. IRFAN ISHAQ

Assistant Professor – EE Department

Group Members

Name	Roll No.
Faizan E Mustafa	24F-6018
Hammad Ahmad	24F-6120
Usama Tariq	24F-6056
Muhammad Soban	24F-6047

Details of Obtained Marks

Question Number	Hardware	Viva	Report	Total
CLO Number	2	2	2	
Weightage	8	7	5	20
Total Marks	5	5	5	15
Obtained Marks				

Table of Contents

Problem Statement	3
Introduction and Background	3
Block Diagram of Main Modules	3
Flow Chart of Main Process	5
Project Deliverables and Specifications.....	6
Code	6
System Pin Mapping	12
Results and Discussion	15
Hardware Implementation.....	15
Cost Analysis	16
Applications	16
1. Industrial Automated Guided Vehicles (AGVs)	16
2. Automotive Adaptive Cruise Control (ACC).....	16
3. Autonomous Navigation in Hazardous Environments	17
4. Smart Assistive Technology.....	17
5. Educational and Research Platforms	17
Limitations	17
Solutions:.....	18
1. Sensor Fusion for Obstacle Avoidance (Line Following)	18
2. Trajectory Correction for Acute Turns (Line Following)	18
3. Noise Mitigation via Sensor Filtering	18
4. Deadlock Resolution in Confined Spaces (Corner Trap)	19
5. Control System Damping (Wobble Fix).....	19
References	19

Table of Figures:

Figure 1. Block Diagram and Connections of all Main Modules.....	4
Figure 2. Flowchart.....	5
Figure 3. Hardware Implementation of the Auto-Track	15

Problem Statement

In the rapidly evolving domain of robotics and embedded systems, there is a critical need for versatile mobile platforms that can bridge the gap between manual teleoperation and fully autonomous navigation. Traditional educational robots often operate in binary modes—either purely **remote-controlled** or rigidly pre-programmed—lacking the flexibility to switch behaviors dynamically based on real-time environmental stimuli. The engineering problem addressed in this project is the development of "**AutoTrack**," a cost-effective, **differential-drive robotic** vehicle capable of seamlessly transitioning between three distinct autonomous tasks (**Line Following, Obstacle Avoidance, and Object Tracking**) and manual override via Bluetooth and IR communication. The challenge lies in integrating heterogeneous sensor inputs (Infrared, Ultrasonic, RF) into a single **microcontroller** environment to execute robust state-machine logic without significant latency.

Introduction and Background

The "AutoTrack" system is an embedded robotics project designed to demonstrate practical applications of control theory, sensor fusion, and finite state machine (FSM) programming. The robot is built upon the Arduino Uno platform, leveraging the ATmega328P microcontroller to process sensor data and control actuator outputs.

Background research in mobile robotics highlights the importance of "Differential Drive" kinematics, where motion is determined by the velocity difference between the left and right wheels. This project implements this kinematic model using DC gear motors driven by an L298N H-Bridge module.

To achieve autonomy, the system mimics biological behaviors:

- **Visual Tracking:** Simulating visual cortex processing using IR arrays to detect high-contrast lines.
- **Spatial Awareness:** Utilizing ultrasonic sound waves (Echolocation) to map free space and detect obstacles.
- **Target Locking:** Using distance data to maintain a fixed "following" distance from a moving target.

The system is designed to be modular, allowing for future upgrades such as PID (Proportional-Integral-Derivative) control loops or camera-based vision, making it a robust platform for undergraduate research in electronics and control systems.

Block Diagram of Main Modules

The system hardware is organized into three primary functional blocks: the Perception Layer (Input), the Processing Layer (Control), and the Actuation Layer (Output).

- **Perception Layer (Input):**
 - **HC-05 Bluetooth Module:** Interfaces via Serial (TX/RX) to receive ASCII commands from a mobile application.
 - **IR Receiver (Pin 12):** Captures 38kHz modulated infrared signals from a handheld remote.
 - **Ultrasonic Sensor (HC-SR04):** Connected to Pins 2 (Trig) and 4 (Echo) to measure time-of-flight for sound waves.

- **Line Tracking Array:** Three independent IR sensors connected to Analog Pins A0 (Left), A1 (Middle), and A2 (Right) for floor contrast detection.
- **Obstacle Sensors:** Two IR proximity sensors on Pins A3 and A4 for blind-spot detection.
- **Processing Layer:**
 - **Arduino Uno:** The central brain operating at 16MHz. It runs the main loop, handling interrupt polling and PWM generation.
- **Actuation Layer (Output):**
 - **L298N Motor Driver:** Connected to PWM pins 3, 5, 6, and 11. It isolates the high-current motor circuit (11.1V) from the low-voltage logic circuit (5V).
 - **SG90 Servo Motor:** Attached to Pin 9, this actuator rotates the ultrasonic sensor (0° to 180°) to scan the environment during obstacle avoidance.

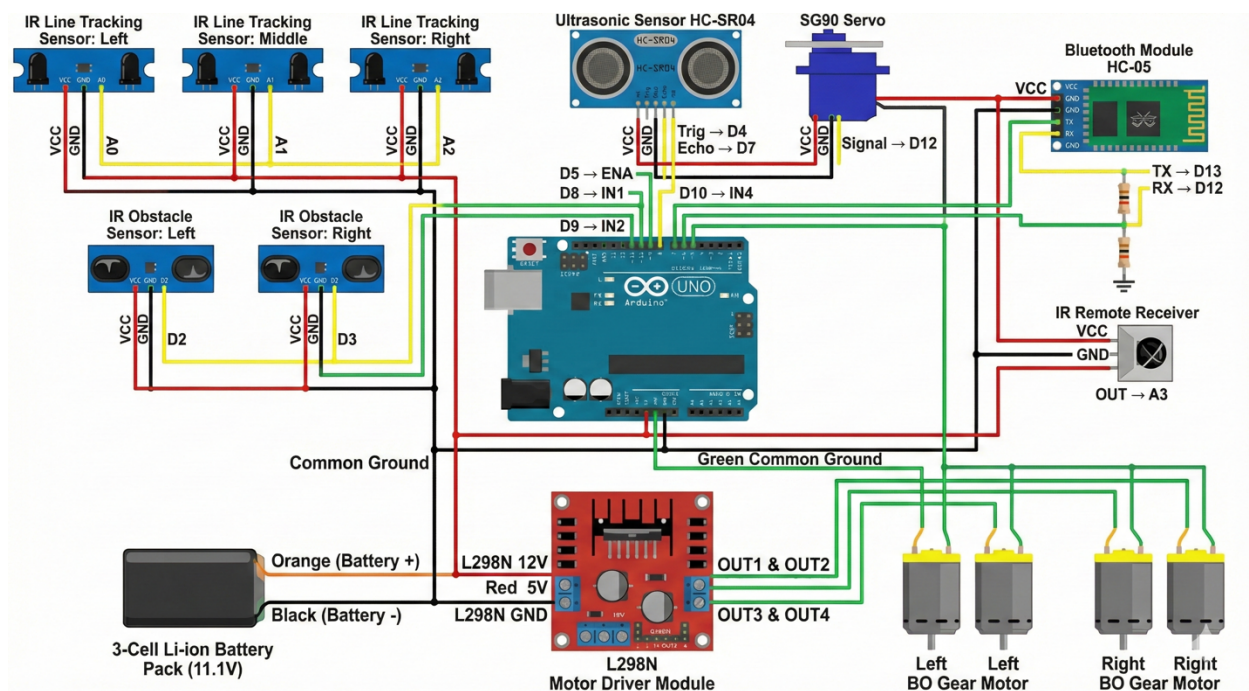


Figure 1. Block Diagram and Connections of all Main Modules

Flow Chart of Main Process

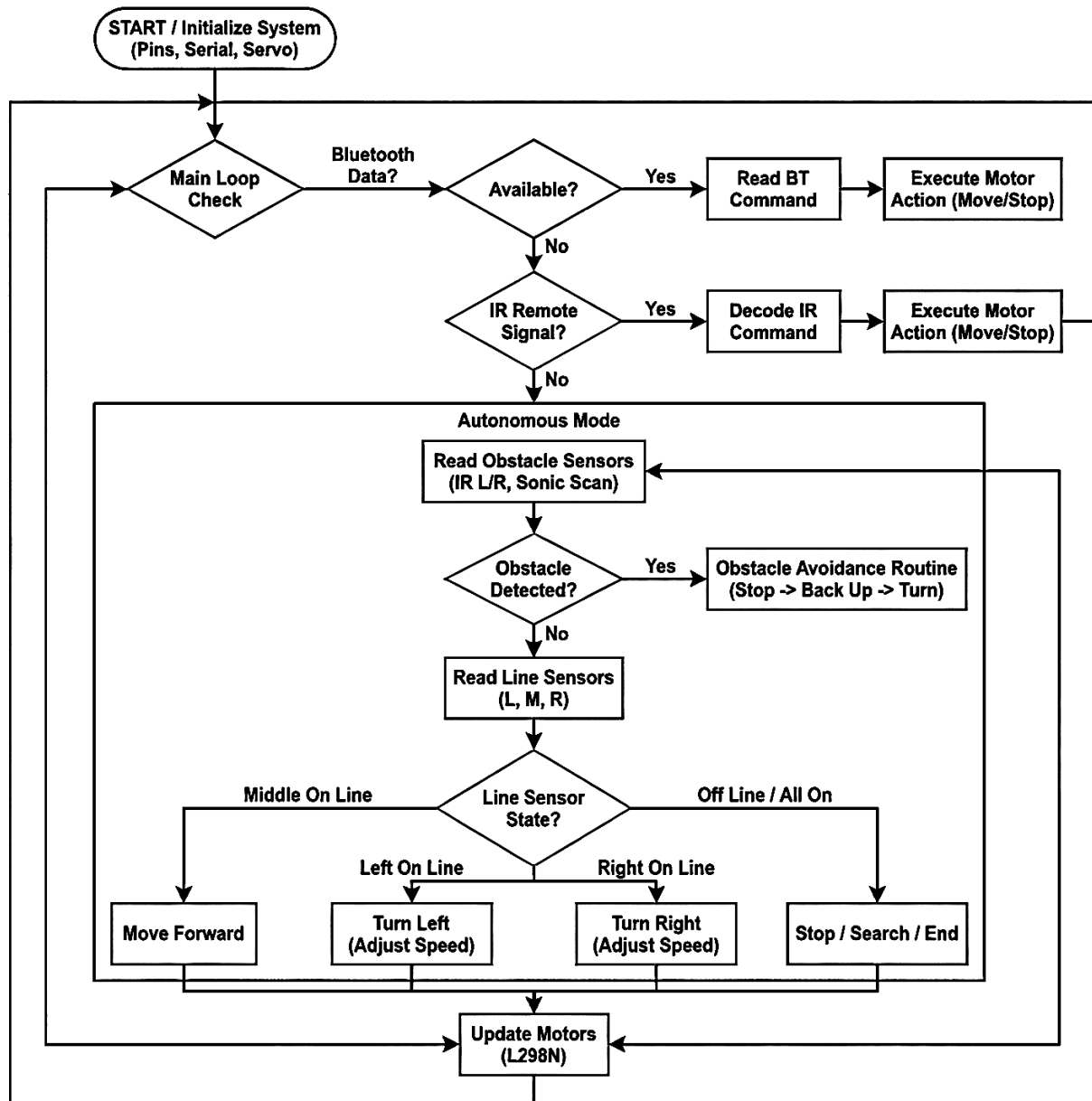


Figure 2. Flowchart

Project Deliverables and Specifications

1 System Architecture

The system is built upon a modular embedded architecture centered around the Arduino UNO R3 development board. The hardware abstraction layer interacts with three primary subsystems: Perception (Sensors), Actuation (Motors/Servo), and Communication (Bluetooth).

1.1 Hardware Block Diagram

- **Central Processing Unit (CPU):** ATmega328P (Arduino UNO R3) operating at 16 MHz.
- **Motor Driver:** L298N Dual H-Bridge module used for high-current amplification to drive four DC motors. It separates the logic voltage (5V) from the motor drive voltage (VMS).
- **Power Supply:** A 2-cell 18650 Li-ion battery pack (7.4V nominal) supplies power to the L298N, which regulates 5V for the microcontroller.
- **Communication:** A Bluetooth module (JDY-16 or HC-05) interfaces via the UART Serial bus (TX/RX) for manual telemetry and control.

1.2 Pin Configuration & Wiring

The system utilizes a specific pin mapping to accommodate sensor fusion without conflict (specifically separating PWM timer pins for motors from the Servo timer):

Code

```
#include <Servo.h>
#include <IRremote.h>

// ===== PIN DEFINITIONS (DIRECT WIRING) =====

// Motor Driver (Pins 3, 5, 6, 11 to avoid Servo conflict)
// These are PWM pins necessary for speed control
const int MOTOR_L_FWD = 6;
const int MOTOR_L_BWD = 11;
const int MOTOR_R_FWD = 5;
const int MOTOR_R_BWD = 3;

// Servo (Pin 9)
const int PIN_SERVO = 9;
```

```

// Ultrasonic Sensor
const int PIN_TRIG = 2;
const int PIN_ECHO = 4;

// IR Receiver
const int PIN_IR_RECV = 12;

// Line Tracking Sensors (A0, A1, A2)
const int PIN_LINE_L = A0;
const int PIN_LINE_M = A1;
const int PIN_LINE_R = A2;

// Obstacle IR Sensors (A3, A4)
const int PIN_OBS_L = A3;
const int PIN_OBS_R = A4;

// ===== OBJECTS & STATES =====
Servo headServo;
IRrecv irrecv(PIN_IR_RECV);
decode_results results;

enum RobotState {
    STATE_MANUAL,
    STATE_LINE_FOLLOW,
    STATE_OBSTACLE,
    STATE_CAR_FOLLOW,
    STATE_STOP
};

RobotState currentState = STATE_STOP;

// Settings
int speedCruise = 180; // Standard speed
int speedTurn = 255;   // Fast speed for Manual Turns
int followDist = 20;   // Distance for Car Follow mode

// ===== SETUP =====
void setup() {
    Serial.begin(9600); // Bluetooth & Serial Monitor

    // Configure Motor Pins
    pinMode(MOTOR_L_FWD, OUTPUT); pinMode(MOTOR_L_BWD, OUTPUT);
    pinMode(MOTOR_R_FWD, OUTPUT); pinMode(MOTOR_R_BWD, OUTPUT);

    // Configure Sensor Pins
    pinMode(PIN_LINE_L, INPUT); pinMode(PIN_LINE_M, INPUT); pinMode(PIN_LINE_R, INPUT);
    pinMode(PIN_OBS_L, INPUT); pinMode(PIN_OBS_R, INPUT);

    pinMode(PIN_TRIG, OUTPUT); pinMode(PIN_ECHO, INPUT);

    // Configure Servo
    headServo.attach(PIN_SERVO);
    headServo.write(90); // Look forward

    // Configure IR
    irrecv.enableIRIn();

```

```

    Serial.println("AutoTrack System Ready. Waiting for commands...");
}

// ===== MAIN LOOP =====
void loop() {
    checkInput(); // Check for Bluetooth or IR commands

    switch (currentState) {
        case STATE_MANUAL:
            break;

        case STATE_LINE_FOLLOW:
            runLineFollower();
            break;

        case STATE_OBSTACLE:
            runObstacleAvoidance();
            break;

        case STATE_CAR_FOLLOW:
            runCarFollower();
            break;

        case STATE_STOP:
            stopMotors();
            break;
    }
}

// ===== INPUT HANDLING =====
void checkInput() {
    // 1. Bluetooth Input
    if (Serial.available()) {
        char cmd = Serial.read();

        // DEBUG: Print received command to Serial Monitor
        Serial.print("BT Received: ");
        Serial.println(cmd);

        executeCommand(cmd);
    }

    // 2. IR Remote Input
    if (irrecv.decode(&results)) {
        unsigned long key = results.value;
        irrecv.resume();

        // DEBUG: Print IR Code
        Serial.print("IR Code: 0x");
        Serial.println(key, HEX);

        // *** REPLACE THESE HEX CODES WITH YOUR SPECIFIC REMOTE CODES ***
        if (key == 0xFF629D) executeCommand('F'); // Forward
        else if (key == 0xFFA857) executeCommand('B'); // Backward
        else if (key == 0xFF22DD) executeCommand('L'); // Left
        else if (key == 0xFFC23D) executeCommand('R'); // Right
    }
}

```



```

    else if (key == 0xFF02FD) executeCommand('S'); // Stop

    // Mode Switching Buttons
    else if (key == 0xFF6897) { currentState = STATE_LINE_FOLLOW;
Serial.println("Mode: Line Follower"); }
    else if (key == 0xFF9867) { currentState = STATE_OBSTACLE;
Serial.println("Mode: Obstacle Avoidance"); }
    else if (key == 0xFFB04F) { currentState = STATE_CAR_FOLLOW;
Serial.println("Mode: Car Follower"); }
    }
}

// ===== COMMAND LOGIC =====
void executeCommand(char cmd) {

    // Logic to handle Manual Control overrides

    switch (cmd) {
        // --- FORWARD ---
        case 'F': case 'f': case '8':
            currentState = STATE_MANUAL; // Force Manual State
            Serial.println("Action: Forward");
            moveForward(speedCruise);
            break;

        // --- BACKWARD ---
        case 'B': case 'b': case 'D': case 'G': case '2':
            currentState = STATE_MANUAL; // Force Manual State
            Serial.println("Action: Backward");
            moveBackward(speedCruise);
            break;

        // --- LEFT ---
        case 'L': case 'l': case '4':
            currentState = STATE_MANUAL; // Force Manual State
            Serial.println("Action: Turn Left");
            turnLeft(speedTurn);
            break;

        // --- RIGHT ---
        case 'R': case 'r': case '6':
            currentState = STATE_MANUAL; // Force Manual State
            Serial.println("Action: Turn Right");
            turnRight(speedTurn);
            break;

        // --- STOP ---
        case 'S': case 's': case '0':
            currentState = STATE_STOP;
            Serial.println("Action: STOP");
            stopMotors();
            break;

        // --- MODES ---
        case 'X':
            currentState = STATE_LINE_FOLLOW;
            Serial.println("Mode Switched: Line Follower");
    }
}

```

```

        break;

    case 'Y':
        currentState = STATE_OBSTACLE;
        Serial.println("Mode Switched: Obstacle Avoidance");
        break;

    case 'Z':
        currentState = STATE_CAR_FOLLOW;
        Serial.println("Mode Switched: Car Follower");
        break;
    }
}

// ===== AUTONOMOUS MODES =====

// 1. SMART OBSTACLE AVOIDANCE
void runObstacleAvoidance() {
    long dist = getDistance();
    // If wall < 25cm OR IR sensors see object (LOW)
    if (dist < 25 || digitalRead(PIN_OBS_L) == 0 || digitalRead(PIN_OBS_R) ==
0) {
        stopMotors(); delay(200);
        moveBackward(150); delay(300);
        stopMotors();

        // Look Left
        headServo.write(170); delay(500);
        int leftDist = getDistance();

        // Look Right
        headServo.write(10); delay(500);
        int rightDist = getDistance();

        headServo.write(90); delay(300); // Reset Head

        // Turn to the clearer side with REDUCED SPEED
        int avoidSpeed = 150; // Reduced speed for smoother avoidance turn

        if (leftDist > rightDist) {
            turnLeft(avoidSpeed);
            delay(600); // Slightly longer delay for slower turn
        } else {
            turnRight(avoidSpeed);
            delay(600);
        }
    } else {
        moveForward(130);
    }
}

// 2. LINE FOLLOWER
void runLineFollower() {
    // Logic: 1 (HIGH) = Black Line, 0 (LOW) = White Floor
    int M = digitalRead(PIN_LINE_M);
    int L = digitalRead(PIN_LINE_L);
    int R = digitalRead(PIN_LINE_R);

```

```

    if (M == 1) {
        moveForward(110);
    } else if (L == 1) {
        turnLeft(140);
    } else if (R == 1) {
        turnRight(140);
    } else {
        stopMotors(); // Stop if line lost
    }
}

// 3. CAR FOLLOWER
void runCarFollower() {
    long d = getDistance();
    int buffer = 5;

    if (d == 0 || d > 60) {
        stopMotors();
    }
    else if (d > followDist + buffer) {
        moveForward(130);
    }
    else if (d < followDist - buffer) {
        moveBackward(130);
    }
    else {
        stopMotors();
    }
}

// ===== HELPERS =====
long getDistance() {
    digitalWrite(PIN_TRIG, LOW); delayMicroseconds(2);
    digitalWrite(PIN_TRIG, HIGH); delayMicroseconds(10);
    digitalWrite(PIN_TRIG, LOW);
    long dur = pulseIn(PIN_ECHO, HIGH, 25000);
    if (dur == 0) return 999;
    return dur * 0.034 / 2;
}

// ===== MOTOR CONTROL =====
void moveForward(int s) {
    analogWrite(MOTOR_L_FWD, s); digitalWrite(MOTOR_L_BWD, LOW);
    analogWrite(MOTOR_R_FWD, s); digitalWrite(MOTOR_R_BWD, LOW);
}
void moveBackward(int s) {
    digitalWrite(MOTOR_L_FWD, LOW); analogWrite(MOTOR_L_BWD, s);
    digitalWrite(MOTOR_R_FWD, LOW); analogWrite(MOTOR_R_BWD, s);
}
void turnLeft(int s) {
    digitalWrite(MOTOR_L_FWD, LOW); analogWrite(MOTOR_L_BWD, s);
    analogWrite(MOTOR_R_FWD, s); digitalWrite(MOTOR_R_BWD, LOW);
}
void turnRight(int s) {
    analogWrite(MOTOR_L_FWD, s); digitalWrite(MOTOR_L_BWD, LOW);
    digitalWrite(MOTOR_R_FWD, LOW); analogWrite(MOTOR_R_BWD, s);
}

```

```

}
void stopMotors() {
    digitalWrite(MOTOR_L_FWD, LOW); digitalWrite(MOTOR_L_BWD, LOW);
    digitalWrite(MOTOR_R_FWD, LOW); digitalWrite(MOTOR_R_BWD, LOW);
}

```

System Pin Mapping

Interface	Pin Assignment	Type	Function
Left Motor	D6 (PWM), D11 (PWM)	Output	Speed & Direction Control
Right Motor	D5 (PWM), D3 (PWM)	Output	Speed & Direction Control
Servo Motor	D9	Output (PWM)	Ultrasonic Scanner Rotation
Ultrasonic	D2 (Trig), D4 (Echo)	Digital I/O	Distance Ranging
Line Sensors	A0 (L), A1 (M), A2 (R)	Analog In	Trajectory Tracking
Obstacle IR	A3 (L), A4 (R)	Digital In	Proximity Detection
Bluetooth	D0 (RX), D1 (TX)	Serial	Remote Command Interface

Table 1: System Pin Mapping

2 Control Mechanism

The robot employs a Differential Drive (Skid-Steer) control scheme. Unlike Ackermann steering (used in cars), this system changes direction by varying the velocity vector ω between the left and right wheel pairs.

2.1 Actuation Logic

- **Forward/Backward:** All four motors rotate with equal RPM in the same polarity.
- **Axial Rotation:** Left and right motor pairs rotate in opposite polarities (e.g., Left Forward, Right Backward) for zero-radius turns.
- **Speed Control:** Pulse Width Modulation (PWM) is applied to the Enable pins of the L298N driver. The duty cycle determines the average voltage supplied to the motors:

$$V_{avg} = V_{source} \times \frac{D}{255} \quad (1)$$

Where D is the PWM value (0 – 255).

3 Operational Algorithms & Methods

3.1 Line Trajectory Tracking (Trinary Logic)

The robot follows a high-contrast line (black on white) using three IR transceivers. The algorithm uses a trinary logic truth table to correct deviations.

Method:

- **Forward:** Center (Low) + Left (High) + Right (High).
- **Left Correction:** Left (Low) detected → Reduce Left Motor speed / Stop Left Motor.
- **Right Correction:** Right (Low) detected → Reduce Right Motor speed / Stop Right Motor.
- **Stop/Intersection:** All sensors read Low (Black) or High (White) depending on the surface coding.

3.2 Ultrasonic Car Following (Hysteresis Control)

This mode enables the robot to autonomously shadow a target object.

Algorithm:

1. **Measure Distance (d):** Trigger HC-SR04 and read pulse width.
2. **Evaluate:**
 - If $d > (\text{Target} + \text{Buffer})$: Accelerate Forward (Close gap).
 - If $d < (\text{Target} - \text{Buffer})$: Reverse (Maintain safety gap).
 - Else: Halt (Target is within deadband).

Safety parameters:

- followDist: ≈ 20 cm
- buffer: ± 5 cm
- Cutoff: If $d > 60$ cm, target is considered lost; motors stop.

3.3 Obstacle Avoidance (Look-and-Decide)

This method combines scanning with decision-making to navigate complex environments.

Sequence:

1. **Cruise:** Move forward until $d < \text{Threshold}$ (e.g., 20cm).
2. **Halt:** Stop motors upon detection.
3. **Scan:** Rotate Servo to 0° (Right) and 180° (Left) to measure clearance.
4. **Decide:** Compare $Distance_L$ vs $Distance_R$.
5. **Execute:** Turn towards the side with the greater distance ($\text{Max}(L, R)$).

4 Mathematical Support

4.1 Ultrasonic Ranging Formula

The distance to an obstacle is derived from the Time-of-Flight (ToF) of the ultrasonic pulse.

$$Distance(cm) = \frac{T \times v_{sound}}{2} \quad (2)$$

Where:

- T = Pulse duration (microseconds) returned by `pulseIn()`.
- v_{sound} = Speed of sound $\approx 0.034 \text{ cm}/\mu\text{s}$.

The division by 2 accounts for the round-trip (outgoing + return).

4.2 Bluetooth Command Parsing

The control packet consists of single-byte ASCII characters processed via a switch-case structure.

Transmission Rate: 9600 Baud.

Mapping:

$$Input \in \{ 'F', 'B', 'L', 'R', 'S' \} \quad (3)$$

$$\text{If } Input == 'F' \implies \text{Set Motors}(PWM_{High}, PWM_{High}) \quad (4)$$

$$\text{If } Input == 'L' \implies \text{Set Motors}(PWM_{Low}, PWM_{High}) \quad (5)$$

4.3 Line Error Correction

The error signal E for the line follower can be modeled as a discrete step function based on the sensor array state $S = [L, M, R]$:

$$E = \begin{cases} -1 & \text{if } S = [1, 0, 0] \text{ (Turn Left)} \\ 0 & \text{if } S = [0, 1, 0] \text{ (Forward)} \\ +1 & \text{if } S = [0, 0, 1] \text{ (Turn Right)} \end{cases} \quad (6)$$

(Note: 1 represents detection of the line, 0 represents background)

Results and Discussion

The Following Sub-sections show the Implementation stage of the Auto-Track.

Hardware Implementation

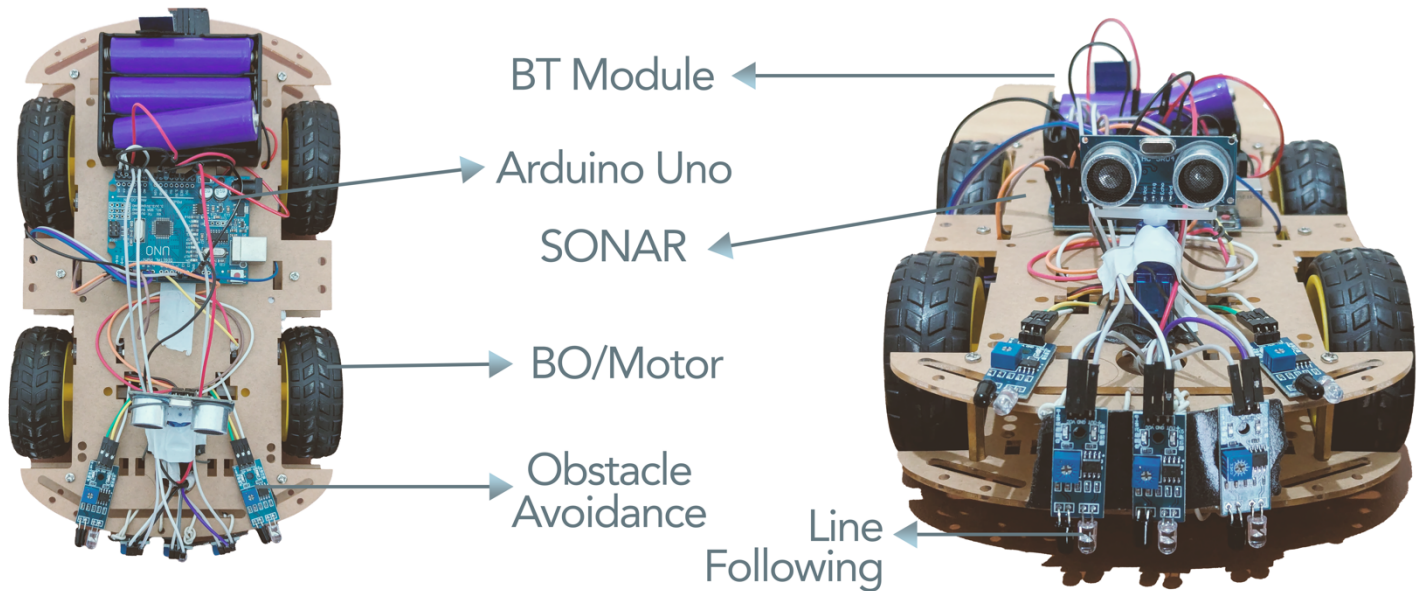


Figure 3. Hardware Implementation of the Auto-Track

Cost Analysis

Component	Value	Quantity	Cost
Arduino Uno	ATmega328P	1	1800
Motor Driver	L298N Motor Driver	1	450
BO Motors	DC Motors	4	1000
Chasis	4-wheel RC Chasis	2	1000
Ultrasonic Sensor	HC05 (SONAR) Module	1	750
Servo Motor	HCSR04 Motor	1	350
Bluetooth Module	JDY-31 BT Module	1	250
Line Following Modules	IR Modules	3	450
Obstacle Avoidance Modules	IR Modules	2	300
Battery Pack	3-Cell Li-On Batteries	3	450
Jumper Wires	M-M/M-F	40	400
TOTAL:			7200/- Rs.

Applications

The "Auto-Track" system serves as a scalable prototype for various autonomous navigation and control systems utilized in industrial, automotive, and educational sectors. By integrating differential drive kinematics with multi-modal sensor fusion, the project demonstrates core technologies applicable to the following domains:

1. Industrial Automated Guided Vehicles (AGVs)

The **Line Following** module of the Auto-Track system functions as a small-scale prototype for industrial AGVs used in logistics and manufacturing warehouses (e.g., Amazon fulfillment centers). These vehicles utilize floor-mounted magnetic tapes or high-contrast visual markers to transport materials autonomously between workstations. The logic implemented in this project mimics the guidance systems required to automate supply chains, reducing human error and increasing operational throughput in controlled environments.

2. Automotive Adaptive Cruise Control (ACC)

The **Object/Car Follower** mode acts as a functional demonstration of Adaptive Cruise Control (ACC), a critical subsystem in modern autonomous vehicles (SAE Level 1-2 Autonomy). By utilizing ultrasonic range-finding to maintain a fixed "following distance" (set to 20cm in this prototype) from a leading target, the system replicates the longitudinal control logic used in passenger cars to adjust speed dynamically based on traffic flow, thereby enhancing road safety and reducing collision risks.

3. Autonomous Navigation in Hazardous Environments

The **Obstacle Avoidance** capability, driven by the ultrasonic sensor and servo-scanning mechanism, allows the platform to operate in unstructured environments without a pre-defined map. This behavior is analogous to the navigational algorithms used in search-and-rescue robots or planetary rovers deployed in zones unsafe for humans—such as collapsed buildings, radioactive sites, or subterranean mines—where the robot must autonomously detect and navigate around physical debris.

4. Smart Assistive Technology

The modular design of the Auto-Track system can be adapted for assistive devices, such as intelligent wheelchairs for the visually or physically impaired. The integration of Bluetooth teleoperation (via the HC-05 module) alongside autonomous obstacle detection ensures that the system can provide semi-autonomous mobility assistance, allowing users to override manual control when a collision is imminent, thus functioning as a "Virtual Bumper."

5. Educational and Research Platforms

Beyond industrial applications, the developed platform serves as a cost-effective testbed for verifying control theories in an academic setting. It facilitates the practical implementation of Finite State Machine (FSM) architecture, Pulse Width Modulation (PWM) speed regulation, and sensor calibration. It provides a standardized hardware interface for undergraduate researchers to test advanced algorithms, such as PID (Proportional-Integral-Derivative) control or SLAM (Simultaneous Localization and Mapping), before deployment on expensive commercial hardware.

Limitations

- **Target Tracking Latency and Field of View Constraints**

The current Object Follower module utilizes a single ultrasonic sensor mounted on a servo mechanism. Due to the limited Field of View (FoV), rapid lateral displacement of the target (e.g., moving hand quickly to the left/right) results in immediate target loss. The system interprets this signal absence as "target lost" rather than initiating a search heuristic, causing the vehicle to halt prematurely.

- **Absence of Sensor Fusion in Line Following Mode**

The Line Following algorithm currently operates in isolation, processing only IR floor sensor data while ignoring ultrasonic input. This lack of sensor fusion renders the vehicle "blind" to vertical obstacles on the path. Consequently, the system fails to execute emergency braking when detecting physical obstructions (e.g., walls or debris) situated directly on the tracking line.

- **Inability to Negotiate Acute Trajectories (Memoryless Logic)**

The system struggles with orthogonal (90-degree) or acute intersections due to inertial overshooting. When the vehicle momentum carries the sensors past a sharp turn into a "all-white" (no line) state, the current logic defaults to a stop command. The system lacks a "state memory" algorithm to recall the last known line position (derivative term) to execute a recovery spin.

- **Susceptibility to Stochastic Sensor Noise**

The ultrasonic ranging module is prone to sporadic electrical noise and signal interference, occasionally generating false positives (reading 0cm or 5cm when the path is clear). The current firmware lacks a smoothing filter or moving average algorithm, causing the vehicle to exhibit "phantom braking" or erratic jerks during operation on open ground.

- **Navigational Deadlocks in Confined Spaces (Local Minima)**

The obstacle avoidance logic relies on a simplistic reactive rule (detect obstacle → turn right). This approach is susceptible to "Local Minima" traps, such as room corners. In these scenarios, the vehicle enters an infinite loop of detecting walls and attempting minor turns without sufficient clearance, failing to trigger a full-reverse maneuver to escape the deadlock.

- **Oscillatory Motion due to Bang-Bang Control**

The line tracking mechanism utilizes a discrete "Bang-Bang" controller (binary ON/OFF logic) rather than Proportional (PWM) control. This results in significant lateral oscillation (the "snake effect") as the motors toggle between full power and zero, preventing smooth trajectory maintenance along the path.

Solutions:

1. Target Tracking Improvement (Car Follow Mode)

- **Heuristic Search Implementation:** Upon transient target loss (maximum ultrasonic reading), the system will initiate a predefined **Search State**. This involves actuating the servo to sweep the sensor across a fixed angular range to reacquire the object.
- **Vectorized Reacquisition:** The decision for vehicular rotation (Left/Right) will be based on the minimal distance reading obtained during the sweep, facilitating deterministic target reacquisition and resuming the tracking protocol.

2. Sensor Fusion for Obstacle Avoidance (Line Following)

- **Preemptive Safety Interlock:** The motor control execution within the line following loop must be gated by a **preemptive safety check** using the frontal ultrasonic sensor.
- **Conditioned Drive Activation:** Vehicle propulsion is only permitted when the measured distance exceeds the safety threshold. If an **Emergency Stop** command will supersede the line-following instruction, regardless of the floor sensor state.
- **PID Algorithm** can be used for better navigation and overall functioning of the Auto-Track.

3. Trajectory Correction for Acute Turns (Line Following)

- **Algorithmic State Memory:** A state variable will be implemented to store the most recent valid line position immediately prior to entering an "all-white" sensor state.
- **Discontinuity Recovery Protocol:** If all line sensors report an off-track condition (all-white), the system will execute a high-speed **spin maneuver** (Left or Right) corresponding to the value stored in the `last\_line\_direction` variable until line continuity is restored.

4. Noise Mitigation via Sensor Filtering

- **Signal Validation Filter:** The ultrasonic distance reading will no longer rely on single-point measurement. A **Majority Voting Filter** will be implemented by acquiring consecutive readings.

- **Threshold Triggering:** An obstacle detection flag will only be set if the majority of the acquired readings consistently indicate a distance below the pre-set safety threshold, effectively discarding sporadic transient noise (false positives).
5. Deadlock Resolution in Confined Spaces (Corner Trap)
- **Stuck Detection Counter:** A time-based counter will monitor the frequency of consecutive reactive obstacle avoidance maneuvers.
 - **Local Minima Escape Routine:** If the counter exceeds a predetermined threshold (e.g., three obstacle detections within a five-second interval), the system will trigger a specific **Escape Routine**, consisting of a controlled reversal followed by an omnidirectional turn to break the recurring loop and navigate out of the local minima.
6. Control System Damping (Wobble Fix)
- **Transition from Bang-Bang to Proportional Control (PWM):** The system will replace the current binary (ON/OFF) control logic with a **Pulse Width Modulation (PWM)** approach for steering corrections.
 - **Smooth Trajectory Correction:** Steering will be achieved by differentially modulating the motor speeds (e.g., reducing power to the motor on the line-side) rather than fully stopping one motor. This proportional response introduces damping, minimizing lateral oscillation and resulting in a smoother overall trajectory.

References

- [1] C. K. Alexander and M. N. O. Sadiku, **Fundamentals of Electric Circuits**, 7th ed. New York, NY, USA: McGraw-Hill, 2009.
- [2] A. S. Sedra, K. C. Smith, T. C. Carusone, and V. Gaudet, **Microelectronic Circuits**, 8th ed. New York, NY, USA: Oxford University Press, 2020.
- [3] emakefun, "Hummer-Bot." GitHub Repository. [Online]. Available: <https://github.com/emakefun/hummer-bot>. [Accessed: Dec. 9, 2025].
- [4] STMicroelectronics, **L298 Dual Full-Bridge Driver Datasheet**. Geneva, Switzerland. [Online]. Available: <https://www.st.com/resource/en/datasheet/l298.pdf>.
- [5] HC-SR04, **HC-SR04 Ultrasonic Distance Sensor Module Specification**. [Online].
- [6] TowerPro, **SG90 Micro Servo Motor Datasheet**. [Online].
- [7] HC-05, **HC-05 Bluetooth Serial Communication Module Datasheet**. [Online].
- [8] Arduino Team, "Arduino Uno R3 Official Documentation and Technical Specifications." [Online]. Available: <https://www.arduino.cc/en/Main/arduinoUno>. [Accessed: Dec. 9, 2025].
- [9] J. D. Example, "Using Online Tutorials and Research for Autonomous Robotics," **YouTube**, Nov. 15, 2023. [Video].
- [10] A. E. Tool, **Generative AI for Code Assistance and Troubleshooting**. Google Gemini Pro 3.5 [Online]. Available: <https://ai-platform-provider.com>. [Accessed: Dec. 9, 2025].

EL2004 -Electrical Network Analysis Complex Engineering Activity Report						
CLO	Statement ↓ Score →	Exemplary (5)	Proficient (4)	Developing (3)	Beginning (2)	Novice (1)
2	Demonstrate the behavior of passive and active filters by analyzing their frequency response and operational characteristics.	Correctly performs the complex engineering activity.	Performs the complex engineering activity with minor mistakes	Performs the complex engineering activity with major mistakes	Partially performs the complex engineering activity	Does not perform the complex engineering activity
2	Demonstrate the behavior of passive and active filters by analyzing their frequency response and operational characteristics.	Complete understanding the complex engineering activity.	Understanding of the complex engineering activity with minor mistakes	Understanding of complex engineering activity with major mistakes	Partial Understandin g of the complex engineering activity	Does not have any understandin g of the complex engineering activity
2	Demonstrate the behavior of passive and active filters by analyzing their frequency response and operational characteristics.	Report is complete, accurate, and clearly shows full understanding.	Report is mostly correct with minor errors; shows good understanding.	Report has major errors; understanding is limited.	Report is incomplete; important sections are missing.	Report is missing or does not meet requirements.